INSTITUT TEKNOLOGI DEL

POLYMORPHISM & ABSTRACTION

Fakultas Vokasi | D4TRPL | Institut Teknologi Del

Week 5 Sesi 1 - CDN - Tahun Ajaran 2026/2027

Konten

- Concept of Polymorphism
  - Method Overloading
  - Method Overriding
- Concept of Abstract Class





Polymorphism

Apa sebenarnya Polimorfisme?

- Arti kata polimorfisme adalah sesuatu seperti **"satu nama, banyak bentuk"**.
- Polimorfisme di Java diwujudkan dalam bentuk **beberapa method yang memiliki nama yang sama**.
  - Dalam beberapa kasus, beberapa method memiliki nama yang sama, tetapi daftar argumen formal yang berbeda (*overload method: compile-time polymorphism*).
  - Dalam kasus lain, beberapa method memiliki nama yang sama, tipe return yang sama, dan daftar argumen formal yang sama (*override method: runtime polymorphism*).



Bentuk Polymorphism

Dari sudut pandang pemrograman praktis, polimorfisme muncul dalam tiga bentuk berbeda di Java, yaitu:

- *Overloading Method*: Di dalam satu kelas yang sama.
  - `Class Kalkulator memiliki method tambah(int a, int b) dan tambah(double a, double b).`
- *Overriding Method via Inheritance*: Antar kelas dalam hierarki pewarisan (parent-child).
  - `Method bergerak() di class Kendaraan mungkin mencetak "Kendaraan bergerak", tetapi di class Mobil kita override method tersebut menjadi "Mobil bergerak dengan 4 roda".`
- *Overriding Method via Interface*: Ketika kelas mengimplementasikan interface, ia wajib memberikan tubuh method untuk method abstract yang dideklarasikan di interface tersebut. Ini juga merupakan bentuk polimorfisme.

```
// OVERRIDING interface method dari Interface BisaMengangkut
@Override
public void setir() {
   System.out.println("Menyetir " + merk + " dengan stir");
}
```

Method Overloading

Sebuah kelas dapat memiliki **dua atau lebih method dengan nama yang sama jika mereka berbeda dalam parameter** (berbeda jumlah parameter, berbeda tipe parameter, atau keduanya).

1. Overloading dengan mengubah jumlah parameter
2. Overloading method dengan mengubah tipe data parameter

```
class Kalkulator {
  // Satu nama method 'tambah', tapi
  int tambah(int a, int b) { // parameter berbeda
     return a + b;
  }
  double tambah(double a, double b) {  // tipe data parameter berbeda: double
     return a + b;
  }
  int tambah(int a, int b, int c) {
     return a + b + c;
  }
}
```

Method Overloading

Contoh Overloading by Parameter Count

```java
import java.util.*;

public class Polymorphism {
    private static void display(int x){
        System.out.println("Arguments: " + x);
    }

    private static void display(int x, int y){
        System.out.println("Arguments: " + x + " and " + y);
    }

    public static void main(String[] args) {
        display(91); // Memanggil method pertama
        display(14, 38); // Memanggil method kedua
    }
}
```

Output:

```
Arguments: 91
Arguments: 14 and 38
```

Penjelasan:

- Kelas Polymorphism memiliki dua method display yang di-overload:
  - display(int x): Hanya menerima satu parameter integer.
  - display(int x, int y): Menerima dua parameter integer.
- Saat main dijalankan, compiler Java secara otomatis memilih method yang mana akan dipanggil berdasarkan jumlah argumen yang diberikan. Ini adalah compile-time polymorphism karena keputusan ini dibuat saat kode dikompilasi.

Method Overloading

Contoh Overloading by Parameter Type

```
import java.util.*;

public class Polypmorphism {

    private static void display(int x){
        System.out.println("Ini adalah Integer");
    }

    // method ini menerima objek String
    private static void display(String x){
        System.out.println(x + " Ini adalah String");
    }

    public static void main(String[] args) {
        display(1); // Memanggil display(int x)
        display("Hello"); // Memanggil display(String x)
    }
}
```

Output:

```
Ini adalah Integer
Hello Ini adalah String
```

Penjelasan:

- Overloading dilakukan dengan mengubah tipe data parameter.
- Kedua method display memiliki jumlah parameter yang sama (satu), tetapi tipe datanya berbeda: int dan String.
- Compiler lagi-lagi memilih method berdasarkan tipe data argumen yang diberikan saat pemanggilan.

Method Overloading

Bukan Overloading, Jika: Hanya Return Type Berbeda

```
import java.util.*;

public class MethodOverloadingSample {
    public void myMethod(int i) {
        System.out.println("Value passed to me: " + i);
    }
    public int myMethod(int i) { // ERROR: Hanya return type yang berbeda
        return (i + 1);
    }
    public static void main(String args[]) {
        MethodOverloadingSample mos = new MethodOverloadingSample();
        mos.myMethod(633); // Ambigu: Panggil myMethod yang void atau int?
        int k = mos.myMethod(k); // Error juga
        System.out.println("Value returned from myMethod: " + k);
    }
}
```

```
                                                    ^
2 errors
error: compilation failed
```

Kode ini error saat dikompilasi. Mengapa?

Method Overriding

Overriding terjadi dalam konteks **pewarisan (inheritance)**. Sebuah *subclass* dapat mengubah atau memperluas perilaku method yang diwarisi dari parent class-nya. Syarat utama overriding adalah **signature method harus persis sama** (nama, tipe dan urutan parameter, serta tipe return yang *compatible*).

- Tujuannya adalah untuk mencapai ***runtime polymorphism***, di mana method yang dijalankan ditentukan oleh tipe objek sebenarnya (bukan tipe variabel referensinya) saat program berjalan.

- Ketika sebuah method dalam subclass memiliki nama yang sama, parameter atau signature yang sama, dan tipe return yang sama (atau sub-tipe) dengan method dalam super-class-nya, maka method dalam subclass tersebut dapat dikatakan meng-override method dalam super-class.

Aturan Overriding Method

Method **overriding** memiliki aturan ketat untuk overriding, yaitu:

1. **Signature Sama**: Nama, parameter, dan return type harus match atau sesuai.
2. **Akses Modifier**: Access modifier di subclass harus sama atau lebih longgar dari parent. Misalnya, jika superclass: protected, subclass bisa: protected atau public, tetapi tidak boleh private.
3. **Exception**: Method overriding tidak boleh melempar *checked exception* baru yang lebih umum daripada yang dideklarasikan di method parent. Ia boleh melempar *exception* yang sama, subclass-nya, atau tidak melempar *exception* sama sekali.

Method Overriding

Contoh Overriding via Inheritance

```java
import java.util.*;

class OverridingParent {
    public int myMethod(int a, int b) {
        return (a + b); // Implementasi parent: penjumlahan
    }
}

class OverridingChild extends OverridingParent {
    // Overriding method myMethod
    public int myMethod(int k, int y) {
        return (k * y); // Implementasi child: perkalian
    }
}

public class Polymorphism {
    public static void main(String args[]) {
        OverridingParent op = new OverridingParent();
        System.out.println("myMethod on Parent Class: " + op.myMethod(2, 21)); // 23

        OverridingChild oc = new OverridingChild();
        System.out.println("myMethod on Child Class: " + oc.myMethod(2, 21)); // 42
    }
}
```

Output:

```
myMethod on Parent Class: 23
myMethod on Child Class: 42
```

Penjelasan:

- *OverridingParent* memiliki method myMethod() yang menjumlahkan dua angka.
- *OverridingChild* mewarisi *OverridingParent* dan meng-override method myMethod untuk mengalikan dua angka.
- Di main, kita lihat bahwa memanggil myMethod() pada objek *OverridingParent* menghasilkan penjumlahan, sedangkan pada objek *OverridingChild* menghasilkan perkalian. **Ini menunjukkan bahwa subclass telah mengganti perilaku method parent-nya**.

Method Overriding

Contoh Overriding via Interface

```
import java.util.*;

interface MyInterfaceA {
    public void m1();
}

interface MyInterfaceB extends MyInterfaceA {
    public void m2();
}

class MyClassA implements MyInterfaceB {
    public void m1() { // Overriding m1 dari MyInterfaceA
        System.out.println("m1 from MyClassA");
    }
    public void m2() { // Overriding m2 dari MyInterfaceB
        System.out.println("m2 from MyClassA");
    }
}

class MyClassB implements MyInterfaceB {
    public void m1() { // Overriding m1 dari MyInterfaceA
        System.out.println("m1 from MyClassB");
    }
    public void m2() { // Overriding m2 dari MyInterfaceB
        System.out.println("m2 from MyClassB");
    }
}
```

```
// Method Overriding melalui Interfaces
public class runtimePolymorphism {
    public static void main(String args[]) {
        MyInterfaceB mib; // Variabel reference bertipe interface

        mib = new MyClassA(); // mib merujuk ke objek MyClassA
        mib.m1(); // Memanggil m1 dari MyClassA
        mib.m2(); // Memanggil m2 dari MyClassA

        mib = new MyClassB(); // mib sekarang merujuk ke objek MyClassB
        mib.m1(); // Memanggil m1 dari MyClassB
        mib.m2(); // Memanggil m2 dari MyClassB
    }
}
```

Output:

```
m1 from MyClassA
m2 from MyClassA
m1 from MyClassB
m2 from MyClassB
```

Program ini adalah contoh *runtime polymorphism* yang powerful:

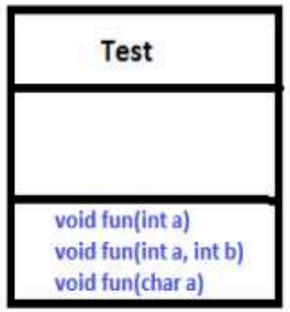
- Variabel mib bertipe MyInterfaceB. Ia dapat menunjuk ke objek dari kelas mana pun yang mengimplementasikan MyInterfaceB (baik MyClassA atau MyClassB).
- Saat mib.m1() dipanggil, JVM akan mengeksekusi implementasi m1() dari objek sebenarnya yang ditunjuk oleh mib pada saat runtime.
- Inilah yang membuat kode kita fleksibel dan mudah diperluas.

Method Overriding

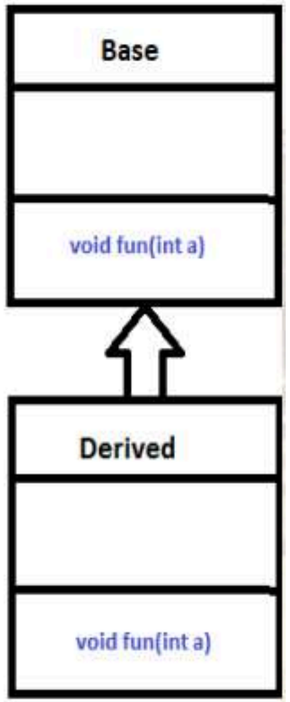
Beberapa method tidak bisa/tidak bermakna untuk di-override:

1. **final method**: Diberi keyword final untuk mencegah overriding.
2. **static method**: Milik kelas, bukan objek. Jadi, tidak di-override tetapi dihidden (jika subclass membuat static method dengan signature yang sama).
3. **private method**: Hanya visible di kelasnya sendiri, sehingga tidak diwarisi dan tidak bisa di-override.
4. **Constructor**: Bukan method warisan, jadi tidak ada konsep overriding constructor.
5. **Abstract method**: Justru harus di-override oleh subclass konkretnya.

Overriding vs Overloading



Overloading



Overriding

Overriding Method vs Overloading

- **Overloading adalah** tentang method yang sama memiliki signature yang berbeda. **Overriding adalah** tentang method yang sama, signature yang sama, tetapi kelas yang berbeda yang terhubung melalui pewarisan.

- **Overloading adalah** contoh polimorfisme waktu kompilasi (*compile-time polymorphism*) dan **overriding adalah** contoh polimorfisme waktu jalankan (*runtime polymorphism*).

Abstract Class

Abstraksi adalah proses menyembunyikan detail implementasi dan hanya menunjukkan fungsionalitas kepada pengguna.

- Sebuah kelas yang dideklarasikan sebagai abstract dikenal sebagai ***abstract class***.
- Kelas abstrak dapat memiliki method abstrak dan non-abstrak. Kelas abstrak perlu diperluas (*extend*) dan methodnya diimplementasikan. Kelas abstrak tidak dapat diinstansiasi (tidak bisa dibuat objeknya langsung).

Penjelasan:

Kelas Abstrak adalah kelas yang tidak lengkap. Ia bertindak sebagai kerangka dasar (*blueprint*) untuk kelas-kelas turunannya.

Kita tidak bisa membuat objek langsung dari kelas abstrak (new AbstractClass(): akan error).

Tujuannya adalah untuk memaksakan suatu struktur hierarki dan mewajibkan subclass untuk mengimplementasikan method-method tertentu (yaitu method abstrak).

Abstract Class

Poin-poin yang perlu diingat mengenai **kelas abstrak**:

- **Keyword**: Gunakan *abstract* sebelum *class*.
- **Kombinasi Method**: Boleh berisi method biasa (dengan implementasi) dan method abstrak (tanpa implementasi, hanya deklarasi).
- **Tidak bisa new**: Langsung membuat objek dari kelas abstrak adalah error.
- **Bisa punya Constructor**: Constructor kelas abstrak bisa dipanggil oleh constructor subclass (biasanya via super()) ketika objek subclass dibuat.
- **Final Method**: Jika sebuah method di kelas abstrak dinyatakan final, maka subclass tidak bisa meng-override method tersebut.

Abstract Class

Contoh Kelas Abstrak Sederhana:

```java
import java.util.*;

abstract class Bike {
    abstract void run(); // Method abstrak (tanpa body)
}

class bicycle extends Bike { // bicycle harus mengimplementasikan run()
    void run() { // Implementasi method abstrak
        System.out.println("Ayok Jalan");
    }

    //Ingat! main method harus berada dalam kelas yang dapat diinstansiasi, misalnya di sini
    public static void main(String args[]) {
        Bike obj = new bicycle(); // Polimorfisme: referensi Bike, objek bicycle
        obj.run(); // Memanggil run() yang diimplementasikan di bicycle
    }
}
```

Penjelasan:

- Bike adalah kelas abstrak dengan satu method abstrak run().
- Kelas bicycle (seharusnya Bicycle sesuai konvensi) mewarisi Bike dan wajib memberikan implementasi untuk method run().
- Di main, kita membuat objek bicycle tetapi menyimpannya dalam variabel referensi bertipe Bike. Ini adalah polimorfisme.
- Saat obj.run() dipanggil, JVM menjalankan implementasi run() dari kelas bicycle.

Output:

```
Ayok Jalan
```

Abstract Class

Contoh Kelas Abstrak dengan Interface:

```java
import java.util.*;

interface A {
    void a();
    void b();
    void c();
    void d();
}

abstract class B implements A { // Kelas abstrak yang implement interface
    public void c() { // Hanya mengimplementasikan method c() dari A
        System.out.println("Ini C");
    }
    // Method a(), b(), d() tidak diimplementasikan di sini (jadi abstract secara implisit)
}

class M extends B { // Kelas konkret M harus implement sisa method abstrak
    public void a() {
        System.out.println("Ini A");
    }
    public void b() {
        System.out.println("Ini B");
    }
    public void d() {
        System.out.println("Ini D");
    }
}
```

Output:

```
Ini A
Ini B
Ini C
Ini D
```

```java
public class abstraction {
    public static void main(String args[]) {
        A a = new M(); // Polimorfisme menggunakan interface
        a.a();
        a.b();
        a.c(); // Method c() diimplementasikan di kelas abstrak B
        a.d();
    }
}
```

Penjelasan:

Program ini menunjukkan interaksi antara interface, kelas abstrak, dan kelas konkret.

1. Interface A mendeklarasikan 4 method.
2. Kelas Abstrak B mengimplementasikan A. Ia hanya memberikan implementasi untuk satu method, c(). Karena B abstrak, ia tidak harus mengimplementasikan semua method dari A. Method a(), b(), dan d() tetap menjadi tanggung jawab subclass-nya.
3. Kelas Konkret M mewarisi B. Karena M konkret, ia wajib mengimplementasikan semua method abstrak yang masih tersisa, yaitu a(), b(), dan d().
4. Di main, objek M dirujuk oleh variabel bertipe A. Pemanggilan method akan diarahkan ke implementasi yang sesuai, baik yang ada di B (c()) maupun di M (a(), b(), d()).

Quick Excercise: (1)

```java
import java.util.*;

class Vehicle {
    public void start() {
        System.out.println("Vehicle is starting");
    }
}

class Car extends Vehicle {
    public void start() {
        System.out.println("Car is starting with key");
    }

    public void start(String remote) {
        System.out.println("Car is starting with " + remote);
    }
}

public class Test {
    public static void main(String[] args) {
        Vehicle v1 = new Vehicle();
        Vehicle v2 = new Car();
        Car c1 = new Car();

        v1.start();
        v2.start();
        c1.start();
        c1.start("remote control");

        // v2.start("remote"); // Baris ini di-comment
    }
}
```

Pertanyaan:

1. Apa output yang dihasilkan ketika kode di atas dijalankan?
2. Mengapa pemanggilan v2.start("remote") (pada baris terakhir yang di-comment) akan menyebabkan error kompilasi?
3. Konsep polimorfisme apa (overloading/overriding) yang ditunjukkan pada kode tersebut dan di mana terjadinya?

Quick Excercise: (2)

Soal: Berdasarkan konsep kelas abstrak yang telah dipelajari, lengkapi kode program dengan keterangan berikut:

1. Implementasikan abstract class Employee dengan properti dan method yang sesuai
2. Implementasikan kedua subclass (FullTimeEmployee dan PartTimeEmployee)
3. Buat objek dan demonstrasikan polimorfisme di method main
4. Apa yang terjadi jika kita mencoba membuat objek langsung dari class Employee?

```
import java.util.*;

// 1. Buat abstract class Employee dengan atribut name dan abstract method calculateSalary()
// 2. Buat subclass FullTimeEmployee yang menghitung gaji sebagai basicSalary + allowance
// 3. Buat subclass PartTimeEmployee yang menghitung gaji sebagai hoursWorked * hourlyRate

public class PayrollSystem {
    public static void main(String[] args) {
        // 4. Buat objek FullTimeEmployee dan PartTimeEmployee
        // 5. Tampilkan nama dan gaji masing-masing karyawan
    }
}
```

Penutup

Any Questions?

TERIMAKASIH

Stay Happy & Healthy